

Lab 3 Program: Dockerfile for a Node.js App

Final Folder Structure

Program-3/

→Dockerfile

→src/index.js

→package.json

→package-lock.json

→node_modules

Step 1: Creating a Dockerfile

Create a folder Program-3 and inside the directory create a Dockerfile

mkdir Program-3

cd Program-3

```
1rv24mc012_akanksha@akanshasPC:~$ mkdir Program-3
```

```
1rv24mc012_akanksha@akanshasPC:~$ cd Program-3
```

```
1rv24mc012_akanksha@akanshasPC:~/Program-3$ nano Dockerfile
```

nano Dockerfile

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN mkdir -p dist && cp -r src/* dist/
FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app/package*.json ./
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node","dist/index.js"]
```

Step 2: Create the code

Create a src directory inside which create the index.js file

nano src/index.js

```
const express=require('express');
const app=express();
const PORT=3000;

app.get('/',(req,res)=>{
res.send('Hello');
});

app.listen(PORT,()=>{
console.log('The app is running on port {PORT}');
});
```

Step 3: Create the package.json file

Run npm init -y (this creates the basic package.json).

Manually edit the package.json file to add the “build” and “start” under the “scripts” section and “express” in “dependencies” section. Also edit “main” section to “dist/index.js”.

Step 4: Install dependencies

npm install

Step 5: Build the docker image

docker build -t program-3 .

```
1rv24mc012_akanksha@akanshasPC:~/Program-3$ npm install
added 69 packages, and audited 70 packages in 6s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
1rv24mc012_akanksha@akanshasPC:~/Program-3$ sudo docker build -t program-3 .
[sudo] password for 1rv24mc012_akanksha:
[+] Building 11.1s (14/14) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 376B                                0.0s
=> [internal] load metadata for docker.io/library/node:20-alpine  5.1s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [internal] load build context                                   0.1s
=> => transferring context: 2.31MB                                    0.1s
=> [builder 1/6] FROM docker.io/library/node:20-alpine@sha256:6178e78b97 0.0s
=> CACHED [builder 2/6] WORKDIR /app                               0.0s
=> [builder 3/6] COPY package*.json ./                             0.1s
=> [builder 4/6] RUN npm install                                   2.7s
=> [builder 5/6] COPY . .                                          1.5s
=> [builder 6/6] RUN mkdir -p dist && cp -r src/* dist/           0.3s
=> [stage-1 3/5] COPY --from=builder /app/package*.json ./        0.1s
=> [stage-1 4/5] COPY --from=builder /app/dist ./dist             0.2s
=> [stage-1 5/5] COPY --from=builder /app/node_modules ./node_modules 0.2s
=> exporting to image                                              0.1s
=> => exporting layers                                              0.1s
=> => writing image sha256:dba93a5d2ea3b8b423e9781c577b4777addce5a3fb894 0.0s
=> => naming to docker.io/library/program-3                       0.0s
```

Step 6: Run the docker container

Once the build is completed, run your new image as a container.

```
docker run -p 3000:3000 --name prg3 program-3
```

```
1rv24mc012_akanksha@akanshasPC:~/Program-3$ sudo docker run -it -p 3000:3000 --name prg3 program-3
The app is running on port {PORT}
```

Step 7: Verify the application

Go to the browser and navigate to <http://localhost:3000>

